

SE 3170: Module #6 Lab
Distributed System Testing
The ATM Machine and the Utility Company

In this assignment you will build and test an ATM machine.

You will need to review data persistence (creating and writing to a file that will keep the data after you stop your program. Some supplemental material is provided in Module #6

The ATM machine will allow user access to their bank accounts and manage their money transfer and bill payment. The bank has the following rules:

1- Logging on to the Bank:

- The user has access to their bank ATM using a username and password
 - i. For simplicity, you can hardcode the following username (**se3170user**) and password (**summer2025**) as global variables in the bank log on.
 - ii. When you run your bank ATM, the user will be asked to enter their username and password.
 - iii. Using the **valid** username and password, the user can log on to their bank account then choose either the checking or saving accounts.

2- Checking Account:

- The user can have one checking account
- The user can deposit up to \$5000 per day to this account using one or more transactions
- The user can withdraw a maximum of \$500 per day using one or more transactions
- The user can transfer any amount of money from their checking account to their saving account
- The user can use this account to pay bills to their utility company
- The user can check the balance in their checking account
- The bank does not allow overdraft; the balance can NOT be negative.

3- Saving Account:

- The user can have one saving account
- The user can deposit up to \$5000 per day using one or more transactions
- The user can NOT withdraw any money from this saving account
- The user can transfer up to \$100 per day from their saving account to their checking account using one or more transactions
- The user can NOT use this account to pay bills
- The user can check the balance in their saving account
- The bank does not allow overdraft; the balance can NOT be negative

4- The Utility Company

- The user can have one utility company account
- The utility company **account** is completely separate from the bank **accounts**, they use different logon and storage files. They run as two independent software artefacts.
- The utility company **cannot** have access to the bank accounts.
- The bank **can** have access to the utility company account, just to send money.
- The bank **cannot** read the utility company's bills information.
- The bank can send the utility company ANY amount of money, regardless of the required bill payment.
- The bank can access the utility company (file) and notify the company that a payment has been made.
 - i. For simplicity, you can hardcode the utility company **file name** into your bank bill payment function. This bill payment function at the bank ATM will simply **add one line in the utility file indicating an amount was paid**.
 - ii. The bank does not read the next **Bill Payment** from the utility company, the bank just sends any amount of money as the user determine. The user is responsible for checking the next **Bill Payment** themselves at the utility company then manually select the amount to be paid to the utility company.
- The user has access to their utility company account using a username and password
 - i. For simplicity, you can do the same as per the bank logon above: Hardcode the following username (e.g. **SE3170User**) and password (**summer2025**) as global variables in the utility accounts.
 - ii. When you run your utility account, the user will be asked to enter their username and password

- iii. Using a **valid** username and password, the user can log on to their utility account and check their bill payment history
 1. The bill payment history will include the last 3-4 bills, each bill will have two lines:
 - a. Bill Date, Bill Amount and Due Date (e.g. January Bill, \$24.00, due January 31)
 - b. Bill Payment and Payment Date (e.g. Payment of \$24.00 on January 25)
- iv. The user can also check if they have unpaid bill amount using “check payment history”
- v. The “check payment history” function will simply display the payment history as below. The user can see if a bill is due, or if all bills were paid.

*January Bill: \$24.00
Due Date: January 31
Payment: \$24.00
Payment Date: January 31*

*February Bill: \$28.00
Due Date: February 28
Payment: \$28.00
Payment Date: February 26*

*March Bill: \$35.00
Due Date: March 31
Payment: \$20.00
Payment Date: March 29*

*April Bill: \$33.00
Due Date: April 30*

5- How Bills Are Paid

- 1- The user will log on to their utility company account and select “check payment history” to see the bill payment history and check for any new bills to pay.
 - i. Remember: The Bank and Utility companies are separate entities, they do not exchange data except for the bank sending money to the utility company.
- 2- The user will then log on to the bank ATM checking account and select “Pay Bills”
- 3- The ATM will ask the user how much money to pay to the utility company.
- 4- The bank will check for sufficient fund in the user’s checking account then deduct the payment amount from the checking account balance and send the money to the utility company (by simply adding two lines to the utility payment file: a “**Payment: \$XX.00**” and a “**Payment Date: Month Day.**” This will update the user payment history at the utility company.)
- 5- The user will be able to see the updated balance on his checking account.
- 6- The user can log on to the utility company and will see **two new lines** added to payment history file indicating the new payment made and the date of that payment.

Deliverables:

- 1- Source code of both the system code and the test scripts
 - a. Use exception handling and add appropriate comments to your code
 - b. Use appropriate persistent storage structures for the bank and the utility company
 - c. Interaction I/O will be done using **command line and keyboard**
 - d. Add a Read-me file of how to deploy and use your assignment
 - e. Use your own software code components
- 2- A comprehensive test plan of your software, including:
 - a. Test designs and **execution** of your code functions and algorithms, storage structures, and user actions for handling the functionalities of their two bank accounts and the utility company (valid and invalid transactions will be covered in your tests)
 - b. For data storage: Test null storage, null element with multiple elements, null single element, incompatible types, empty elements as well as two normal cases (single and multiple elements).
 - c. Test outcomes (screen shots and description must be provided)
 - d. Sufficient transactions for testing
- 3- One Pdf file containing your test designs in deliverable #2 and screenshots of your tests and the corresponding output with description.

